May 21, 2026 · Research

What we've learned building cloud agents

Josh Ma · 9 min read

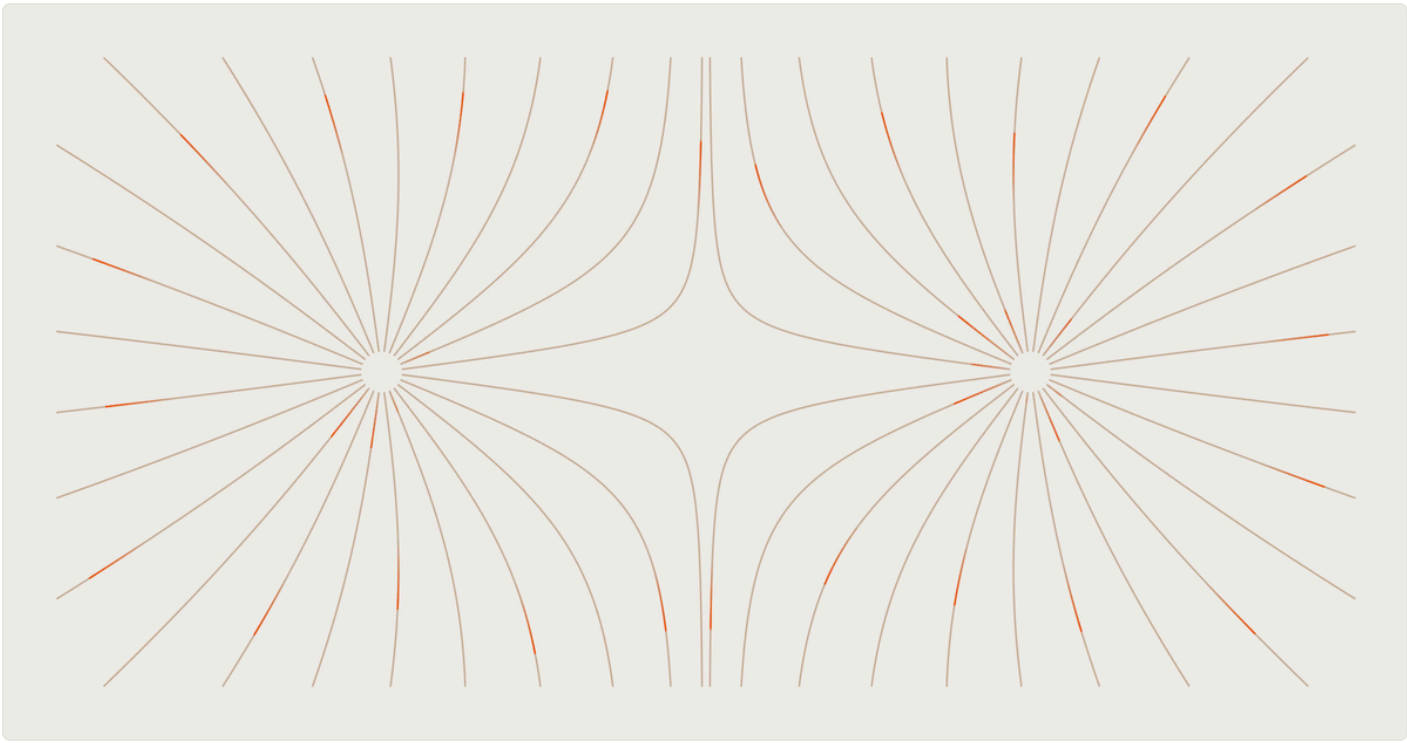


Table of Contents

When we first launched cloud agents a year ago, they seemed like a straightforward extension of local agents. Since then, cloud agent capabilities have expanded considerably.

tasks than a local agent sitting on your laptop.

These capabilities introduce challenges around environment setup, reliability, and orchestration that are less pronounced when an agent is running on your laptop.

In this post, we want to share the biggest lessons we've learned building cloud agents, and why the work increasingly looks less like porting a local agent to a server and more like building an operating layer around it.

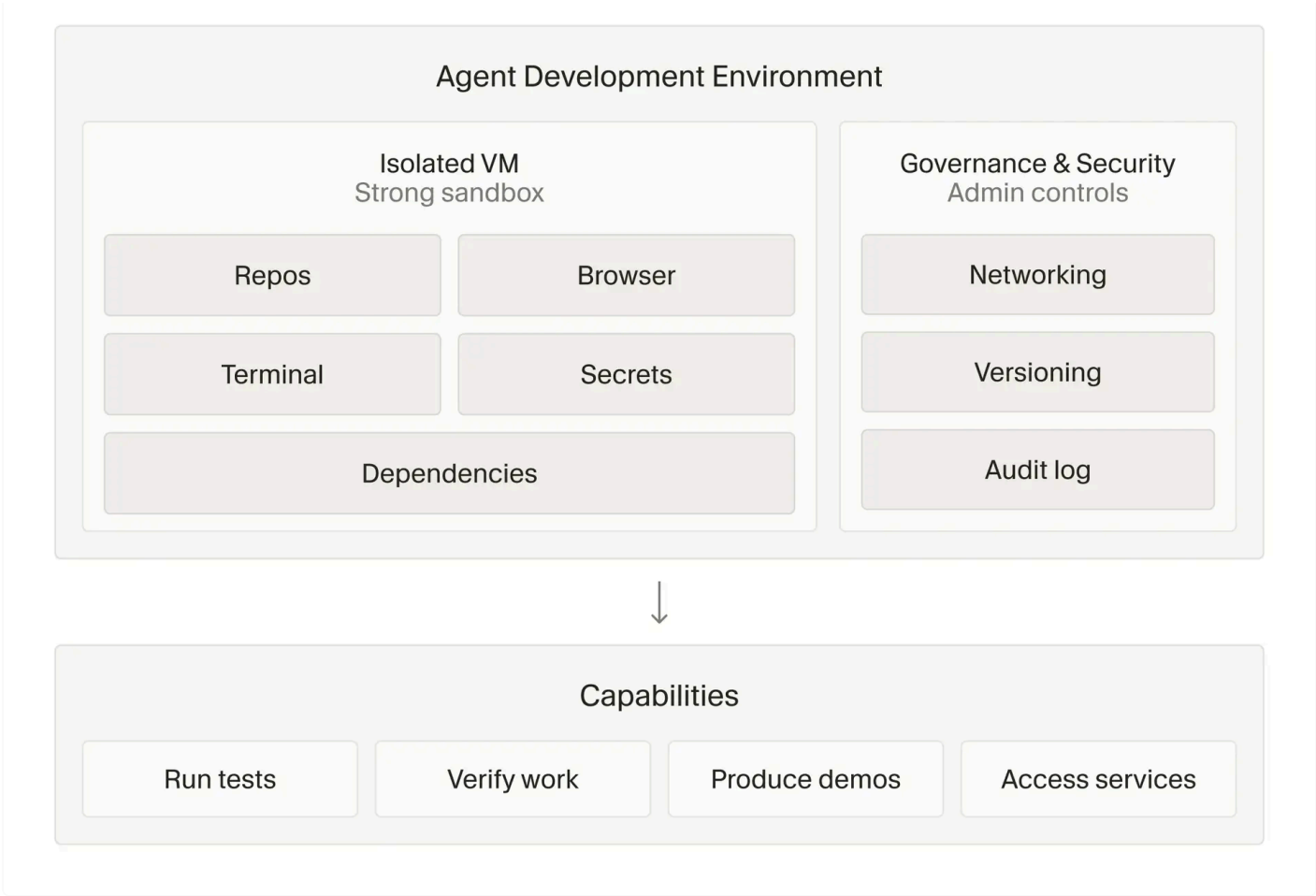
The development environment is the product

Over the last year we've learned that the single biggest factor in cloud agent output quality is ensuring it has a full development environment, like a developer has.

This isn't something you have to think as much about locally, because local agents inherit your working development environment for free, from your laptop. In the cloud, you have to reconstruct all of that from scratch, and it's surprisingly hard to tell when you haven't done it perfectly.

Instead of a crash or an error message, often the only indication is a subtle degradation in output quality. You might not notice it at first, or if you do, you might chalk it up to the model.

But over and over again we've traced it back to the same diagnosis: the cloud agent not having the environment it needs to execute or verify its work. A year ago this mattered less because models couldn't make much use of their environment anyway. But as they've gotten smarter, the environment setup has become the determining factor in whether they execute at their full potential.



Today, getting to "full environment" requires rebuilding a surprising amount of infrastructure:

- Better user tools for building the agent environment
- Methods to efficiently hibernate and resume agent VMs between messages
- Pipelines to quickly and durably checkpoint, restore, and fork VM images
- Tight harness and client integrations so that agents and humans alike can interpret and interact with the environment

And as cloud agents take on more work they need controlled network access to create PRs, pull dependencies, and do research. Over time, we've ended up building what is essentially enterprise IT for agents, complete with secret redaction, network policies, and credential management.

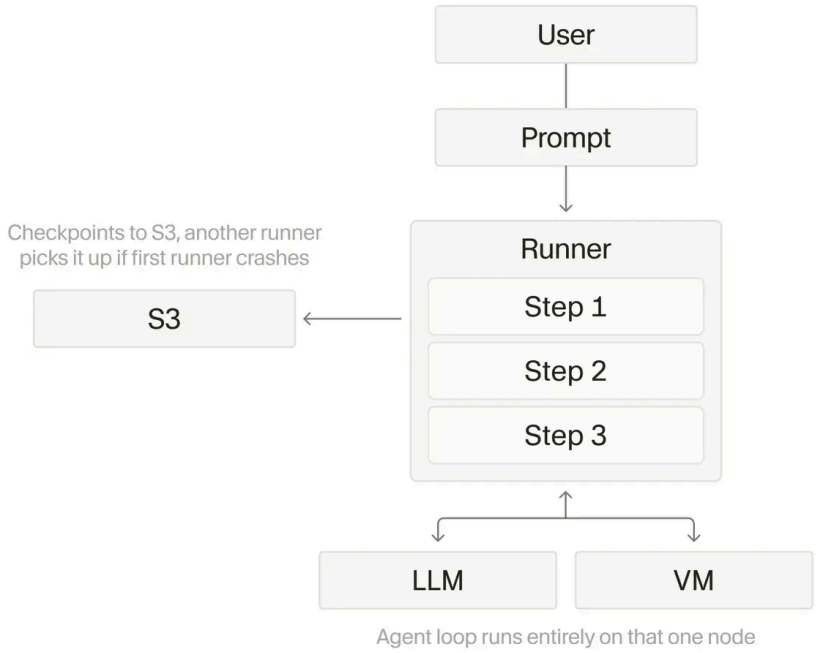
Long-running agents need durable execution

Cloud agents present a different kind of reliability challenge than local agents. Instead of competing for local resources on your laptop, cloud agents run in their own isolated VMs. This makes it easier for

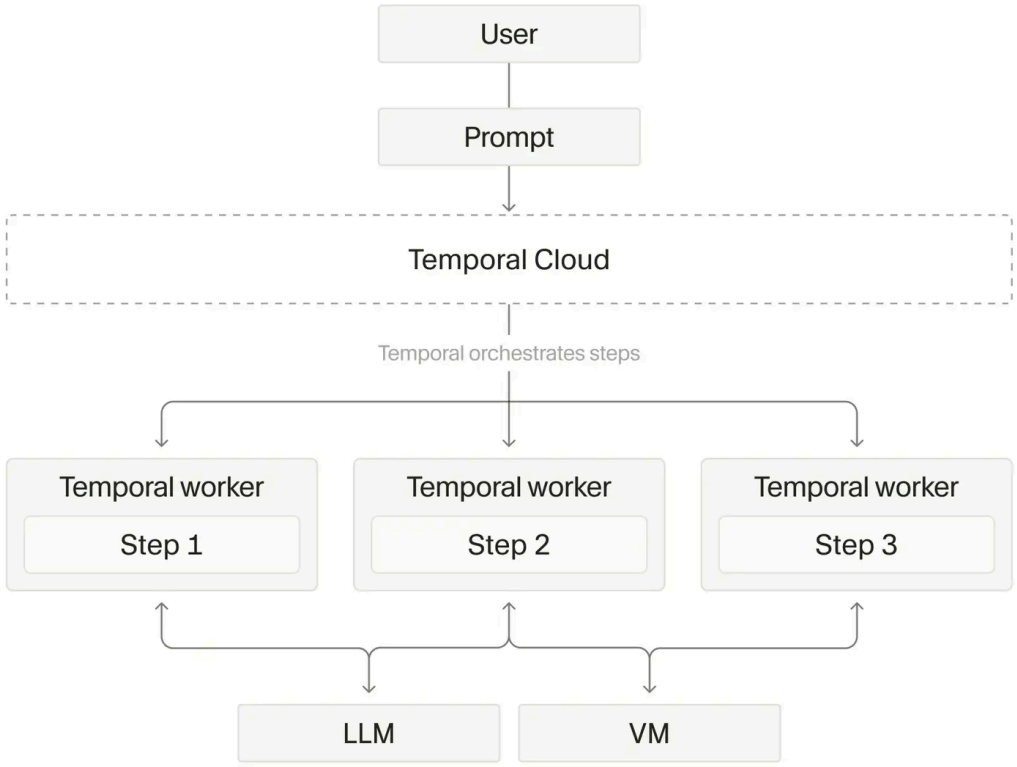
But, running in a VM creates exposure to disruptions like inference provider outages, pods needing to be replaced, and EC2 nodes going down.

We started building cloud agents with a work-stealing architecture, where worker nodes could pick up agents and loop them to completion. It transplanted what works locally to a server and it was a fragile setup—our early beta of cloud agents often operated at one 9 of reliability.

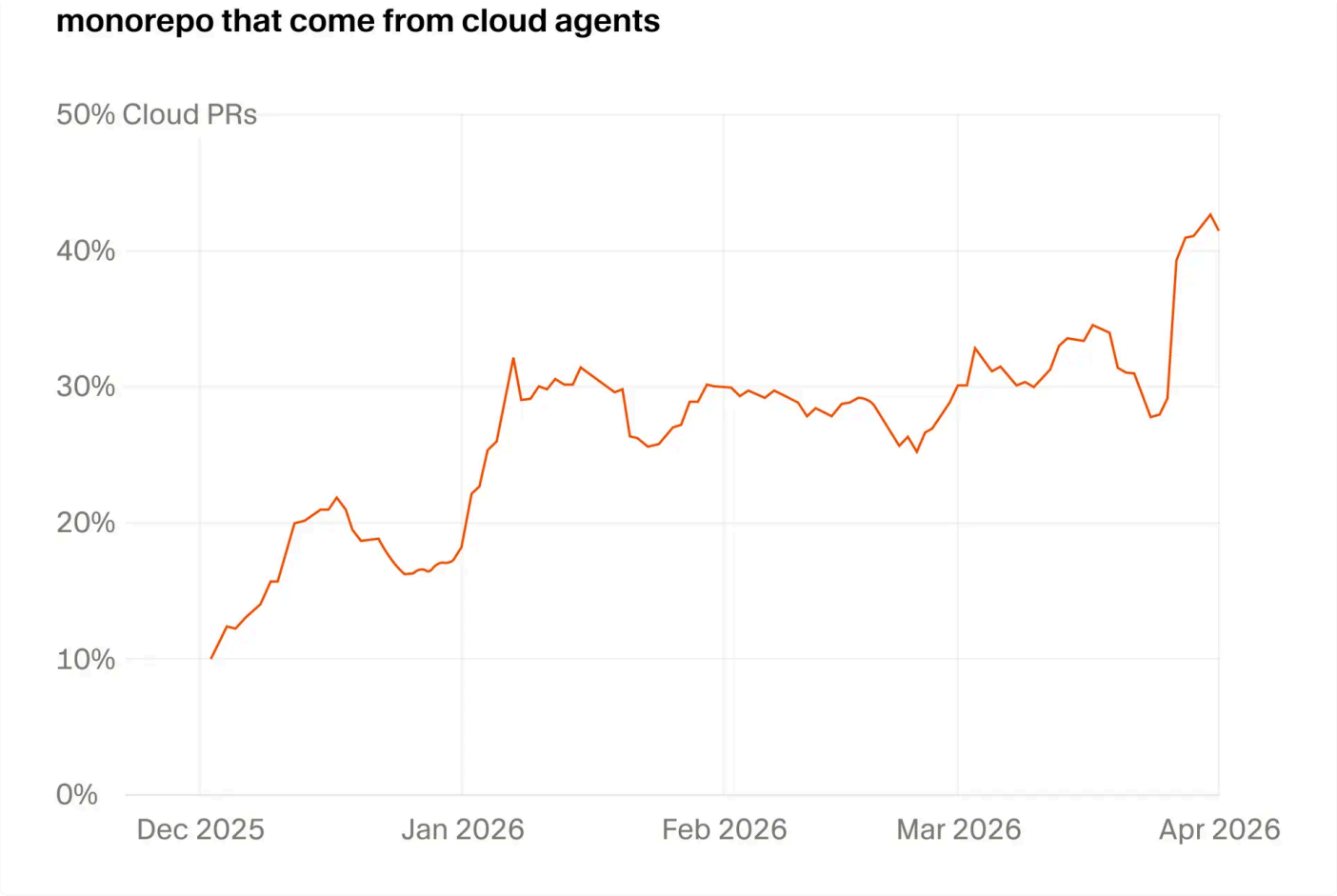
Dedicated runner architecture



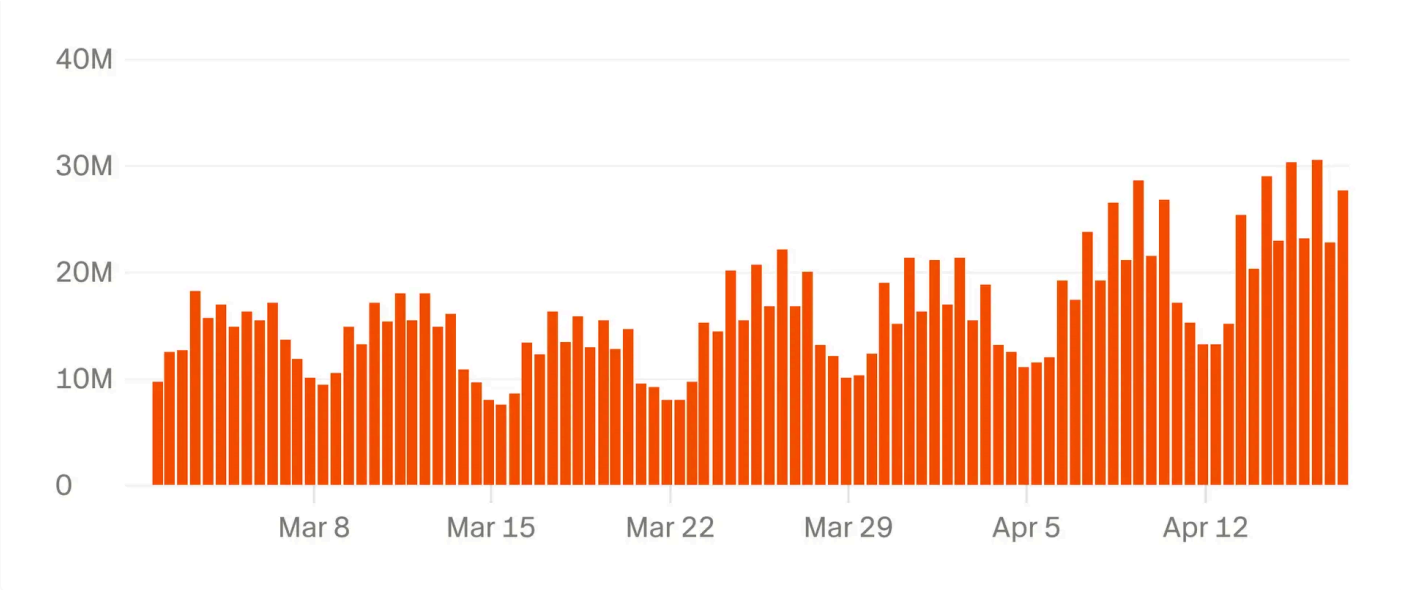
As cloud agents matured, we found ourselves on the verge of rebuilding a lot of the durable execution primitives that Temporal already solves (e.g., retry mechanisms, scheduling work across machines, durability across node failures), so instead we migrated there.



Our current agent loop on Temporal can survive blips in inference reliability, pod hibernation and resumption, and runs that stretch across days or even weeks. That migration alone took us past two 9s of reliability and today, Temporal handles more than 50 million actions per day across more than 7 million unique workflows. Internally, more than 40% of our PRs come from cloud agents, and growing.

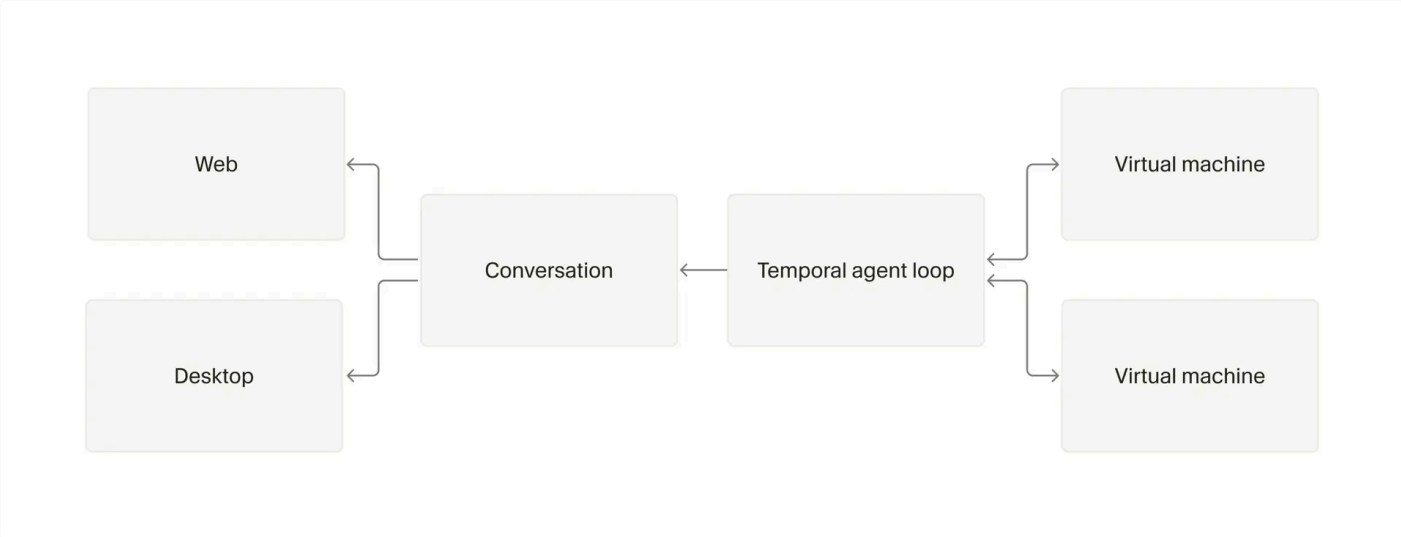


Over time, we've learned how to better architect our Temporal workflows. We've moved from "eternal" agent workflows to multiple shorter ones that exit after completing a single task, which makes version upgrades easier. We've also split out activities to better capture timeouts and retries as async tool calls, subagents, and inference provider outages have changed our underlying assumptions.



Decoupling agents and machines from conversation state

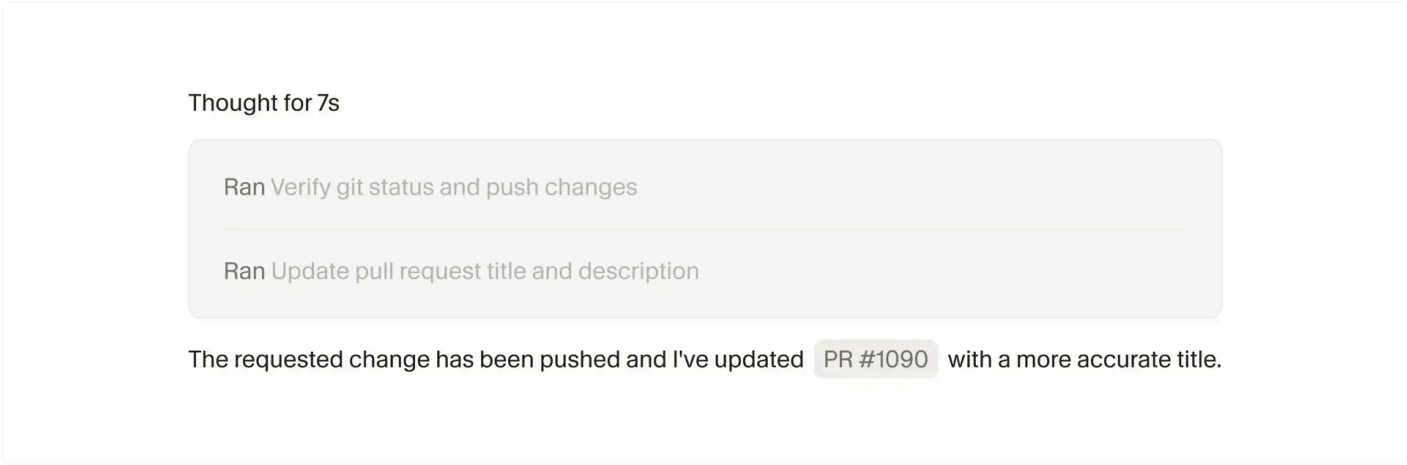
A cloud agent is no longer just one loop running on one machine. Instead, an agent might run on one machine, spawn async subagents across several, or start locally then delegate work to the cloud. A subagent might even outlive its parent, or run on a completely different kind of pod.



To make that work, we've found it valuable to keep the agent loop, the machine state, and the conversation state as decoupled components. Because the agent loop lives in Temporal rather than on the VM itself, we can manage pod lifecycles independently and run agents across different kinds of pods — including optimizations like readonly VMs or prewarmed VMs.

and desktop clients. This layer accounts for retries, so that if a step of the agent loop fails after streaming partial output and then gets retried, the client can detect this, rewind its stream, and show the new data instead of the old.

Knowing how to get out of the way



Building a cloud agent harness means constantly reevaluating how much behavior is deterministic and how much gets handed to the agent.

Early on, we didn't trust the agent very much, so the harness would double-check its work after every task, force a commit, and push. As models got smarter, we started moving logic out of the harness and into tools the agent controls. A year ago, multi-repo setups required hardcoded harness behavior. Now, we can give the agent the repo layout, expose tools for branches and PRs, and let it decide how to do the work.

The same thing happened with CI Autofix, where earlier versions of our cloud agent harness contained logic for grabbing job failure logs and writing them to the VM. Now, we just give the agent access to the GitHub CLI and automatically write large outputs to files it can search through. The notification to the agent got much simpler, and we expect that trend to continue.

The harness isn't going away so much as what it contains is changing. Computer use is a good example right now. Our cloud agent harness has a dedicated subagent type for computer use, with its own model routing, custom prompting, and screen recording. The VNC and Chrome belong to the environment, which is shared between the parent agent and the subagent. This lets the parent make use of them directly, for example, by running a Playwright script. We use this scaffolding because models aren't quite ready to handle computer use on their own, but the agent still controls when to invoke it.

agent has stopped and is waiting for permission, but in the cloud, it could sit there for hours before you go back and check on it.

Related posts

Self-healing agent environments

olding the agent's hand
standing the system

access is blocked, or
and to then be able to
for achieving this which

xpect the rate of change
to only accelerate from here. Cursor cloud agents let teams take advantage of this expansive surface

Author: Josh Ma

Apr 30, 2026 · Research
Continually improving our agent harness
Stefan & Jediah · 11 min read

Apr 21, 2026 · Research
Keeping the Cursor app stable
Andrew & Kevin · 8 min read

Apr 6, 2026 · Research
Better MoE model inference with warp decode
Less, Federico & Zhiyuan · 10 min read

View more posts →